

1. What is EWMA in a Simple Way?

Exponentially Weighted Moving Average (EWMA) is a method to smooth out noisy sequences of numbers (like daily temperatures, stock prices, or mini-batch gradients in deep learning) to reveal the underlying trend.

The formula is:

$$v_t = \beta v_{t-1} + (1 - \beta)\theta_t$$

θ_t : The current raw observation at time step t (e.g., current gradient).

v_t : The running smoothed estimate at time step t .

β : The momentum/decay factor (typically between 0.9 and 0.99).

Intuition:

Think of v_t as a memory bank. β decides how much of the **past memory** you retain (e.g., 90%), and $(1 - \beta)$ decides how much **new information** you let in (e.g., 10%). It effectively averages over approximately $\frac{1}{1-\beta}$ past observations.

2. The Problem with $v_0 = 0$ (The "Cold Start" Issue)

When we start an algorithm (like Adam or Momentum SGD), we typically set $v_0 = 0$.

Let's see what happens at step 1 and step 2 if $\beta = 0.9$ and your first gradient is $\theta_1 = 100$:

Step 1 ($t = 1$):

$$v_1 = 0.9(v_0) + (1 - 0.9)\theta_1 = 0.9(0) + 0.1(100) = 10$$

Step 2 ($t = 2$), assuming $\theta_2 = 100$:

$$v_2 = 0.9(v_1) + 0.1(\theta_2) = 0.9(10) + 0.1(100) = 9 + 10 = 19$$

Even though your true data is around 100, your estimated average starts at 10, then 19, and takes many steps to crawl up to 100. This happens because $v_0 = 0$ pulls the early estimates severely toward zero.

3. Why Don't We Just Initialize $v_0 = \theta_1$?

It seems intuitive to set $v_0 = \theta_1$ to avoid starting at zero. However, doing so introduces major mathematical and practical flaws:

1. It Inverts the Core Principle of EWMA

If you set $v_0 = \theta_1$, let us trace the weights for $\beta = 0.9$:

$$v_1 = 0.9(\theta_1) + 0.1(\theta_1) = \theta_1$$

$$v_2 = 0.9(\theta_1) + 0.1(\theta_2)$$

$$v_3 = 0.9(v_2) + 0.1(\theta_3) = 0.81\theta_1 + 0.09\theta_2 + 0.1\theta_3$$

Look at the weights at $t = 3$:

Weight of the newest point (θ_3): 10%

Weight of the previous point (θ_2): 9%

Weight of the oldest point (θ_1): 81%

Instead of prioritizing recent data, **the initial value θ_1 dominates the entire average for a long time.**

2. Outlier Anchoring

In mini-batch gradient descent, the very first mini-batch gradient (θ_1) might be noisy or an outlier. If you initialize $v_0 = \theta_1$, that initial outlier will linger and distort optimization trajectories for dozens of subsequent steps.

3. Computational and Pipeline Simplicity

In deep learning frameworks (PyTorch, TensorFlow), all parameter buffers (like moments in Adam) are allocated and zero-initialized simultaneously when building the optimizer graph, before any data batches pass through. Initializing with $v_0 = 0$ avoids conditional branch logic during the first step.

4. How Bias Correction Works Technically

To keep $v_0 = 0$ while fixing the early underestimation, we apply **Bias Correction**:

$$\hat{v}_t = \frac{v_t}{1 - \beta^t}$$

Mathematical Derivation:

Unrolling v_t from $v_0 = 0$:

$$v_t = (1 - \beta)\theta_t + \beta(1 - \beta)\theta_{t-1} + \beta^2(1 - \beta)\theta_{t-2} + \dots + \beta^{t-1}(1 - \beta)\theta_1$$

Factoring out $(1 - \beta)$:

$$v_t = (1 - \beta) \sum_{i=1}^t \beta^{t-i} \theta_i$$

To find the sum of all weight coefficients:

$$\text{Sum of Weights} = (1 - \beta) \sum_{i=0}^{t-1} \beta^i = (1 - \beta) \cdot \frac{1 - \beta^t}{1 - \beta} = 1 - \beta^t$$

For an average to be unbiased, the sum of all weights **must equal 1**.

When t is small, the weights sum to $(1 - \beta^t)$, which is strictly less than 1.

Dividing v_t by $(1 - \beta^t)$ rescales the weights so they sum to exactly 1.0.

Testing the Correction on Our Example:

For $\theta_1 = 100, \beta = 0.9$:

Raw value: $v_1 = 10$

Correction term: $1 - \beta^1 = 1 - 0.9 = 0.1$

Corrected estimate:

$$\hat{v}_1 = \frac{v_1}{1 - \beta^1} = \frac{10}{0.1} = 100$$

The estimate is corrected to 100 on step 1.

5. Why the Bias Correction Fades at Larger Timesteps

Because $0 < \beta < 1$, as the number of timesteps t increases, β^t decays exponentially toward 0:

$$\lim_{t \rightarrow \infty} \beta^t = 0 \implies \lim_{t \rightarrow \infty} (1 - \beta^t) = 1$$

Step (t)	$\beta=0.9$	Term: β^t	Divisor: $(1 - \beta^t)$
$t = 1$	0.9^1	0.900	0.100
$t = 2$	0.9^2	0.810	0.190
$t = 10$	0.9^{10}	0.349	0.651
$t = 50$	0.9^{50}	0.005	0.995
$t = 100$	0.9^{100}	≈ 0	≈ 1.000